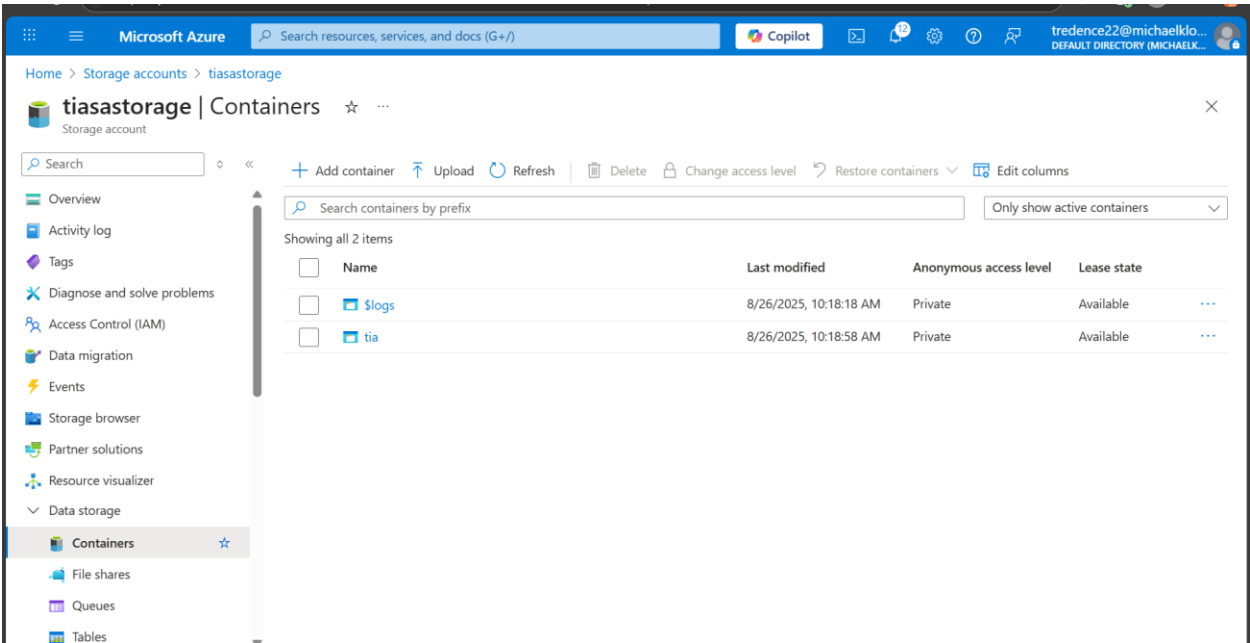


TIASA JANA

EMP ID -7170

CAPSTONE PROJECT-2

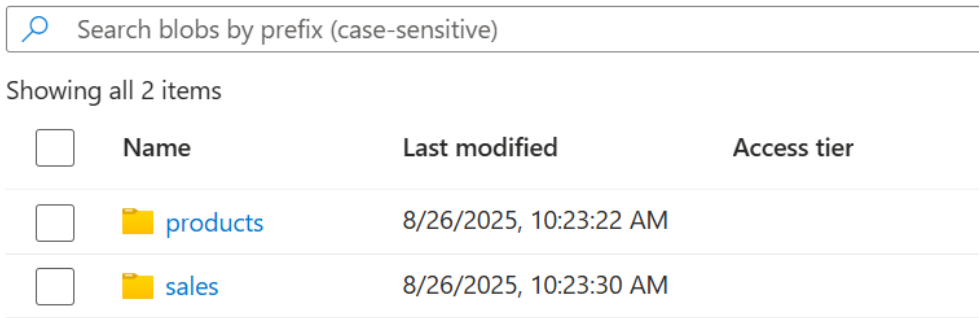
Created Storage container with container



Followed the same folder structure



Authentication method: Access key ([Switch to Microsoft Entra user account](#))



Access Connector Created

AccessConnectorCreate-20250826102804 | Overview

Deployment

Search [x] << [Delete] [Cancel] [Redeploy] [Download] [Refresh]

Overview

- Inputs
- Outputs
- Template

✓ Your deployment is complete

Deployment name : AccessConnectorCreate-20250... Start time : 8/26/2025, 10:28:43 AM
Subscription : [LabsKraft](#) Correlation ID : 2b240a17-876c-4c8e-bc6e
Resource group : [LabsKraft2027](#)

Deployment details

Resource	Type	Status
tiasa_access	Access Connector for Azure Data Lake Storage	Created

Next steps

[Go to resource](#)

Created Catalog with suitable permissions

Catalog Explorer > External Locations > **adls_cap2**

Overview Permissions Browse Workspaces

Description
Add description

Credential
URL
Limit to read-only use

Test connection

Location Type: Directory

- ✓ Success - Read
- ✓ Success - List
- ✓ Success - Write
- ✓ Success - Delete
- ✓ Success - Path Exists
- ✓ Success - Hierarchical Namespace Enabled

All Permissions Confirmed
The associated Storage Credential grants permission to perform all necessary operations.

Done

About this external location
Owner: Tredence LabsKraft22
Fallback mode: ☐



tiasa_catalog4



Share



Create schema

Overview

Details

Permissions

Workspaces

Description



AI generate

Add



Filter schemas

2 schemas

Name

Owner

Created at



default

tredence22@michaelcloudkrafthotm...

Aug 26, 2025, 10:40 AM



information_schema

System user

Aug 26, 2025, 10:40 AM

About this catalog

Owner

Tredence LabsKraft22



Tags

Add tags

Granted full access on silver and bronze layer

Grant on tiasa_catalog4.silver

Granted privileges will be inherited by applicable objects (e.g. tables, views) in this schema. [Learn more](#)

Principals

juhi86516@gmail.com X

Privilege presets

Data Editor (Can read and modify any object in the schema, as well as create new objects)

Prerequisite

☒ USE SCHEMA

Metadata

☒ APPLY TAG

Read

☒ EXECUTE

☒ READ VOLUME

☒ SELECT

Edit

☒ MODIFY

☒ REFRESH

☒ WRITE VOLUME

Create

☒ CREATE FUNCTION

☒ CREATE MATERIALIZED VIEW

☒ CREATE MODEL

☒ CREATE MODEL VERSION

☒ CREATE TABLE

☒ CREATE VOLUME

☒ ALL PRIVILEGES gives all privileges

Cancel Confirm

Catalog Explorer > tiasa_catalog4 >

silver

Overview Details **Permissions**

Grant Revoke Privileges Inherited Type to filter by principal

Principal	Privilege	Object
juhi86516@gmail.com	ALL PRIVILEGES	tiasa_catalog4.silver
juhi86516@gmail.com	EXTERNAL USE SCHEMA	tiasa_catalog4.silver
juhi86516@gmail.com	MANAGE	tiasa_catalog4.silver
techtigress1111@gmail.com	ALL PRIVILEGES	tiasa_catalog4.silver
techtigress1111@gmail.com	EXTERNAL USE SCHEMA	tiasa_catalog4.silver
techtigress1111@gmail.com	MANAGE	tiasa_catalog4.silver

Catalog Explorer > tiasa_catalog4 >

bronze

Overview Details **Permissions**

Grant Revoke Privileges Inherited Type to filter by principal

Principal	Privilege	Object
juhi86516@gmail.com	ALL PRIVILEGES	tiasa_catalog4.bronze
juhi86516@gmail.com	EXTERNAL USE SCHEMA	tiasa_catalog4.bronze
juhi86516@gmail.com	MANAGE	tiasa_catalog4.bronze
techtigress1111@gmail.com	ALL PRIVILEGES	tiasa_catalog4.bronze
techtigress1111@gmail.com	EXTERNAL USE SCHEMA	tiasa_catalog4.bronze
techtigress1111@gmail.com	MANAGE	tiasa_catalog4.bronze

Catalog Explorer > tiasa_catalog4 >

 **gold**    Use with BI tools  Share  Create 

Overview Details Permissions

Grant Revoke Privileges  Inherited  Type to filter by principal 

Principal	Privilege	Object
techtigress1111@gmail.com	ALL PRIVILEGES	 tiasa_catalog4.gold
techtigress1111@gmail.com	EXTERNAL USE SCHEMA	 tiasa_catalog4.gold
techtigress1111@gmail.com	MANAGE	 tiasa_catalog4.gold
juhidataanalyst@gmail.com	EXECUTE	 tiasa_catalog4.gold
juhidataanalyst@gmail.com	READ VOLUME	 tiasa_catalog4.gold
juhidataanalyst@gmail.com	SELECT	 tiasa_catalog4.gold
juhidataanalyst@gmail.com	USE SCHEMA	 tiasa_catalog4.gold

Created 2 external tables in Unity Catalog:-

Products_raw and sales_raw

Catalog Explorer > tiasa_catalog4 >

 **bronze**    Use with BI tools  Share  Create 

Overview Details Permissions

Description

 AI generate Add

Filter tables Tables 2 Volumes 0 Models 0 Functions 0 Sort 

Name	Owner	Created at	Popularity
 products_raw	tredence22@michaelcloud...	Aug 26, 2025, 12:12 PM	
 sales_raw	tredence22@michaelcloud...	Aug 26, 2025, 12:04 PM	

About this schema

Products_raw

```
%sql

-- Select your catalog and schema

USE CATALOG tiasa_catalog4;

USE bronze;


-- Drop the table if it already exists

DROP TABLE IF EXISTS products_raw;


-- Create the external table for products using the discovered schema

CREATE TABLE products_raw (

    category STRING,

    discount_rate DOUBLE,

    price DOUBLE,

    product_id BIGINT,

    product_name STRING

)

USING JSON

OPTIONS (

    multiLine = 'true'

)

LOCATION 'abfss://tia@tiasastorage.dfs.core.windows.net/products/';
```

Sales_raw

```
%sql
```

```
-- Select your catalog and schema
```

```
USE CATALOG tiasa_catalog4;
```

```
USE bronze;
```

```
-- 1. Drop the table only if it already exists to avoid errors
```

```
DROP TABLE IF EXISTS sales_raw;
```

```
-- 2. Create the external table from scratch
```

```
CREATE TABLE sales_raw (
```

```
    sale_id INT,
```

```
    product_id INT,
```

```
    region STRING,
```

```
    store_id STRING,
```

```
    quantity INT,
```

```
    sales_amount DOUBLE,
```

```
    sale_date TIMESTAMP
```

```
)
```

```
USING CSV
```

```
OPTIONS (
```

```
    header = 'true',
```

```
    delimiter = ','
```

```
)
```

```
LOCATION 'abfss://tia@tiasastorage.dfs.core.windows.net/sales/';
```

Products_raw is Queryable

12:12 PM (8s) 10

```
%sql
-- Query the new products table
SELECT * FROM tiasa_catalog4.bronze.products_raw;

-- Join sales with products
SELECT
  s.sale_id,
  p.product_name,
  p.category,
  s.quantity,
  s.sales_amount
FROM
  tiasa_catalog4.bronze.sales_raw s
JOIN
  tiasa_catalog4.bronze.products_raw p ON s.product_id = p.product_id
LIMIT 10;
```

(2) Spark Jobs

_sqlidf: pyspark.sql.connect.dataframe.DataFrame = [sale_id: integer, product_name: string ... 3 more fields]

	sale_id	product_name	category	quantity	sales_amount
1	987	Product_5SQ8	Furniture	6	365.41
2	955	Product_5SQ8	Furniture	6	651.49
3	672	Product_5SQ8	Furniture	2	325.41
4	488	Product_5SQ8	Furniture	20	2163.58
5	289	Product_5SQ8	Furniture	20	1671.28
6	197	Product_5SQ8	Furniture	12	1146.44
7	484	Product_5SQ8	Furniture	5	642.8
8	804	Product_DXS6	Toys	15	2098.04
9	533	Product_DXS6	Toys	15	1782.78
10	404	Product_DXS6	Toys	11	1055.2

Sales_raw is Queryable

12:04 PM (4s) 7

```
%sql
SELECT * FROM tiasa_catalog4.bronze.sales_raw LIMIT 100;
```

(1) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [sale_id: integer, product_id: integer ... 5 more fields]

	sale_id	product_id	region	store_id	quantity	sales_amount	sale_date
1	1	157	APAC	Store_31	16	1289.03	2025-08-11T21:20:34.000+00:...
2	2	65	APAC	Store_9	8	1301.48	2025-08-12T02:40:42.000+00:...
3	3	416	LATAM	Store_25	19	3047.56	2025-08-10T00:09:00.000+00:...
4	4	501	US	Store_29	18	1955.74	2025-07-28T09:49:56.000+00:...
5	5	244	MEA	Store_6	20	2362.98	2025-08-15T00:28:03.000+00:...
6	6	240	EU	Store_22	8	1516.96	2025-08-25T05:58:18.000+00:...
7	7	854	EU	Store_24	10	178.92	2025-07-28T09:55:39.000+00:...
8	8	69	US	Store_9	7	246.4	2025-08-10T22:41:18.000+00:...
9	9	654	EU	Store_3	14	138.23	2025-08-02T05:50:22.000+00:...
10	10	277	US	Store_8	15	145.13	2025-08-15T19:05:21.000+00:...
11	11	584	LATAM	Store_4	13	2516.03	2025-08-08T13:20:55.000+00:...
12	12	432	APAC	Store_1	13	241.39	2025-07-27T22:24:44.000+00:...
13	13	797	US	Store_25	11	1914.19	2025-08-05T00:57:54.000+00:...
14	14	64	MEA	Store_32	15	2545.81	2025-08-09T06:41:32.000+00:...
15	15	256	US	Store_15	18	1482.75	2025-07-31T09:45:00.000+00:...

100 rows, 1.441s runtime

- **Build a DLT pipeline** (retail_sales_pipeline) with Bronze → Silver → Gold tables.
 - **Bronze:** Raw ingestion (no transformations).
 - **Silver:** Cleaned data (remove nulls, normalize columns).
 - **Gold:** Business aggregates (e.g., daily revenue per region, product performance).
- **Write DLT pipeline definitions in Python.**

```
import dlt
```

```
from pyspark.sql.functions import col, expr
```

```
from pyspark.sql.types import (
```

```
    StructType,
```

```
    StructField,
```

```
    IntegerType,
```

```
    StringType,
```

```
    DoubleType,
```

```
        TimestampType,

        LongType

    )

# =====

# STEP 1: DEFINE THE SCHEMAS

# All schema definitions are placed at the top of the notebook.

# =====

# Schema for your raw sales data

sales_schema = StructType([

    StructField("sale_id", IntegerType(), True),

    StructField("product_id", IntegerType(), True),

    StructField("region", StringType(), True),

    StructField("store_id", StringType(), True),

    StructField("quantity", IntegerType(), True),

    StructField("sales_amount", DoubleType(), True),

    StructField("sale_date", TimestampType(), True)

])

# Schema for your raw product data

products_schema = StructType([

    StructField("category", StringType(), True),

    StructField("discount_rate", DoubleType(), True),

    StructField("price", DoubleType(), True),

    StructField("product_id", LongType(), True),

    StructField("product_name", StringType(), True)

])

# =====

# STEP 2: DEFINE THE DATA SOURCES

# =====
```

```
sales_data_path = "abfss://tia@tiasastorage.dfs.core.windows.net/sales/"
```

```
products_data_path = "abfss://tia@tiasastorage.dfs.core.windows.net/products/"
```

```
# =====
```

```
# STEP 3: BUILD THE PIPELINE (BRONZE -> SILVER -> GOLD)
```

```
# =====
```

```
# ----- BRONZE LAYER (RAW INGESTION) -----
```

```
@dlt.table(name="sales_bronze", comment="Raw sales data from cloud storage.")
```

```
def sales_bronze():
```

```
    return (
```

```
        dlt.read(
```

```
            "cloud_files",
```

```
            path=sales_data_path,
```

```
            format="csv",
```

```
            schema=sales_schema,
```

```
            options={"header": "true"})
```

```
    )
```

```
)
```

```
@dlt.table(name="products_bronze", comment="Raw product data from cloud storage.")
```

```
def products_bronze():
```

```
    return (
```

```
        dlt.read(
```

```
            "cloud_files",
```

```
            path=products_data_path,
```

```
            format="json",
```

```
            schema=products_schema,
```

```
            options={"multiLine": "true"})
```

```
    )
```

```
)
```

```

# ----- SILVER LAYER (CLEANED & ENRICHED) -----

@dlt.table(name="sales_silver", comment="Cleaned sales data with quality checks.")

@dlt.expect_or_drop("valid_sale_id", "sale_id IS NOT NULL")

def sales_silver():

    return (

        dlt.read("sales_bronze")

        .select(

            col("sale_id"),

            col("product_id"),

            col("store_id"),

            col("region"),

            col("quantity"),

            col("sales_amount"),

            col("sale_date").alias("sale_timestamp")

        )

    )

@dlt.table(name="products_silver", comment="Cleaned product data.")

@dlt.expect_or_drop("valid_product_id", "product_id IS NOT NULL")

def products_silver():

    return dlt.read("products_bronze")

# ----- GOLD LAYER (BUSINESS AGGREGATES) -----

@dlt.table(name="daily_revenue_by_region_gold", comment="Aggregated daily revenue by region.")

def daily_revenue_by_region_gold():

    return (

        dlt.read("sales_silver")

        .groupBy("region", expr("to_date(sale_timestamp) as sale_date"))

        .sum("sales_amount")

        .withColumnRenamed("sum(sales_amount)", "total_revenue")

    )

```

```
@dlt.table(name="product_performance_gold", comment="Aggregated sales metrics per product.")
```

```
def product_performance_gold():
```

```
    sales_df = dlt.read("sales_silver")
```

```
    products_df = dlt.read("products_silver")
```

```
    return (
```

```
        sales_df.join(products_df, "product_id", "inner")
```

```
        .groupBy("product_id", "product_name", "category")
```

```
        .agg(
```

```
            expr("sum(quantity) as total_quantity_sold"),
```

```
            expr("sum(sales_amount) as total_revenue")
```

```
        )
```

```
    )
```

Pipeline Created

The screenshot displays the Databricks Jobs & Pipelines interface for a pipeline named **tiasa_26_pipeline**. The pipeline is currently in the **Development** tab and is **Running** as of 8/26/2025, 3:03:20 PM.

Pipeline Details:

- Pipeline ID:** 8d538523-7a28-4ff5-89dd-dc5a009ed70b
- Pipeline type:** ETL pipeline
- Source code:** [/Users/tredence22@michaelkloudkrafthotmail.onmicrosoft.com/tetee](#)
- Run as:** Tredence LabsKraft22
- Tags:** None

Event log:

Time	Category	Message
4 minutes ago	user_action	User tredence22@michaelkloudkrafthotmail.onmicrosoft.com started an update.
4 minutes ago	create_update	Update 2c71b6 started by USER_ACTION.
4 minutes ago	update_progress	Update 2c71b6 is INITIALIZING.

Health of the tables are fine as every table is green as data ingestion and processing is done successfully.

Part 5: Monitoring & Troubleshooting

- Show how to monitor **DLT pipeline health** in the Databricks UI.
- Generate sample errors (e.g., upload a corrupted file) and demonstrate troubleshooting.
- **Expectation:** You must interpret DLT event logs and resolve failures.

Code

```
import dlt

import pyspark.sql.functions as F

from pyspark.sql.types import (
    StructType, StructField,
    StringType, IntegerType, DoubleType, DateType
)

# =====

# BRONZE LAYER - Raw Ingestion (Auto Loader + Rescued Columns)

# =====

@dlt.table(
    name="sales_bronze",
    comment="Bronze: raw incremental sales files via Auto Loader (with rescued data)"
)

def sales_bronze():
    return (
        spark.readStream
            .format("cloudFiles")
            .option("cloudFiles.format", "csv")
```

```

.option("header", "true")

.option("cloudFiles.inferColumnTypes", "true")

.option("cloudFiles.schemaLocation", "abfss://tia@tiasastorage.dfs.core.windows.net/checkpoints/sales_schema")

.option("cloudFiles.schemaEvolutionMode", "addNewColumns")

.option("rescuedDataColumn", "_rescued_data")

.load("abfss://tia@tiasastorage.dfs.core.windows.net/sales")

)

@dlt.table(

name="products_bronze",

comment="Bronze: raw product metadata (with rescued data)"

)

def products_bronze():

return (

    spark.readStream

        .format("cloudFiles")

        .option("cloudFiles.format", "csv")

        .option("header", "true")

        .option("cloudFiles.inferColumnTypes", "true")

        .option("cloudFiles.schemaLocation", "abfss://tia@tiasastorage.dfs.core.windows.net/checkpoints/products_schema")

        .option("cloudFiles.schemaEvolutionMode", "addNewColumns")

        .option("rescuedDataColumn", "_rescued_data")

        .load("abfss://tia@tiasastorage.dfs.core.windows.net/products")

)

# Expose rescued rows for inspection

@dlt.table(name="sales_bronze_rescued", comment="Rescued rows from sales ingestion")

def sales_bronze_rescued():

return dlt.read("sales_bronze").filter(F.col("_rescued_data").isNotNull())

@dlt.table(name="products_bronze_rescued", comment="Rescued rows from products ingestion")

def products_bronze_rescued():

```

```

return dlt.read("products_bronze").filter(F.col("_rescued_data").isNotNull())

# =====

# SILVER LAYER - Cleaned Data + Expectations + Quarantine

# =====

@dlt.table(

    name="tiasa_catalog4.silver.sales_silver",

    comment="Silver: cleaned sales (types, null filters, dedupe)",

    table_properties={"delta.enableChangeDataFeed": "true"}

)

@dlt.expect("Valid sales", "sales_amount >= 0")

@dlt.expect("Valid quantity", "quantity > 0")

def sales_silver():

    df = dlt.read("sales_bronze")

    df2 = (

        df

        .filter(F.col("sale_id").isNotNull() & F.col("product_id").isNotNull())

        .withColumn("quantity", F.col("quantity").cast("int"))

        .withColumn("sales_amount", F.col("sales_amount").cast("double"))

        .withColumn("sale_date", F.to_date(F.col("sale_date"), "yyyy-MM-dd"))

        .withColumn(

            "sales_amount_calculated",

            (F.col("quantity") * (F.col("sales_amount") /

                F.when(F.col("quantity") == 0, F.lit(1)).otherwise(F.col("quantity"))))

            )

        )

    return df2.dropDuplicates(["sale_id"])

# Quarantine table for expectation failures

@dlt.table(

    name="sales_silver_quarantine",

```



```

        comment="Rows failing sales expectations (invalid sales_amount or quantity)"
    )

def sales_silver_quarantine():

    df = dlt.read("sales_bronze")

    return df.filter(

        (F.col("sales_amount").cast("double") < 0) |

        (F.col("quantity").cast("int") <= 0)

    )

@dlt.table(

    name="tiasa_catalog4.silver.products_silver",

    comment="Silver: cleaned products; handles new discount_rate column"

)

def products_silver():

    df = dlt.read("products_bronze")

    df2 = (

        df

        .withColumn("price", F.col("price").cast("double"))

        .withColumn("discount_rate", F.coalesce(F.col("discount_rate").cast("double"), F.lit(0.0)))

    )

    return df2.dropDuplicates(["product_id"])

# =====

# GOLD LAYER - Business Aggregates

# =====

@dlt.table(

    name="tiasa_catalog4.gold.daily_revenue_region",

    comment="Gold: daily revenue aggregated by region"

)

def gold_daily_revenue_region():

    df = dlt.read("tiasa_catalog4.silver.sales_silver")

```

```

return (

    df.groupBy("sale_date", "region")

        .agg(

            F.round(F.sum("sales_amount").cast("double"), 2).alias("daily_revenue"),

            F.count("sale_id").alias("num_transactions")

        )

    )

)

@dlt.table(

    name="tiasa_catalog4.gold.product_performance",

    comment="Gold: total sales per product (with product metadata)"

)

def gold_product_performance():

    sales = dlt.read("tiasa_catalog4.silver.sales_silver")

    products = dlt.read("tiasa_catalog4.silver.products_silver")

    joined = sales.join(products, on="product_id", how="left")

    return (

        joined.groupBy("product_id", "product_name", "category")

            .agg(

                F.round(F.sum("sales_amount").cast("double"), 2).alias("total_sales_amount"),

                F.round(F.avg("discount_rate"), 4).alias("avg_discount_rate"),

                F.count("sale_id").alias("total_transactions")

            )

        )

    )

# =====

# MONITORING TABLE - Expectation Summary

# =====

@dlt.table(

    name="sales_silver_expectation_summary",

    comment="Summary of sales_silver expectation outcomes by batch"

```

)

```
def sales_silver_expectation_summary():
```

```
    df = dlt.read("sales_bronze")
```

```
    return (
```

```
        df.groupBy(F.current_date().alias("as_of_date"))
```

```
        .agg(
```

```
            F.count("*").alias("rows_seen"),
```

```
            F.sum((F.col("sales_amount").cast("double") < 0).cast("int")).alias("neg_sales_rows"),
```

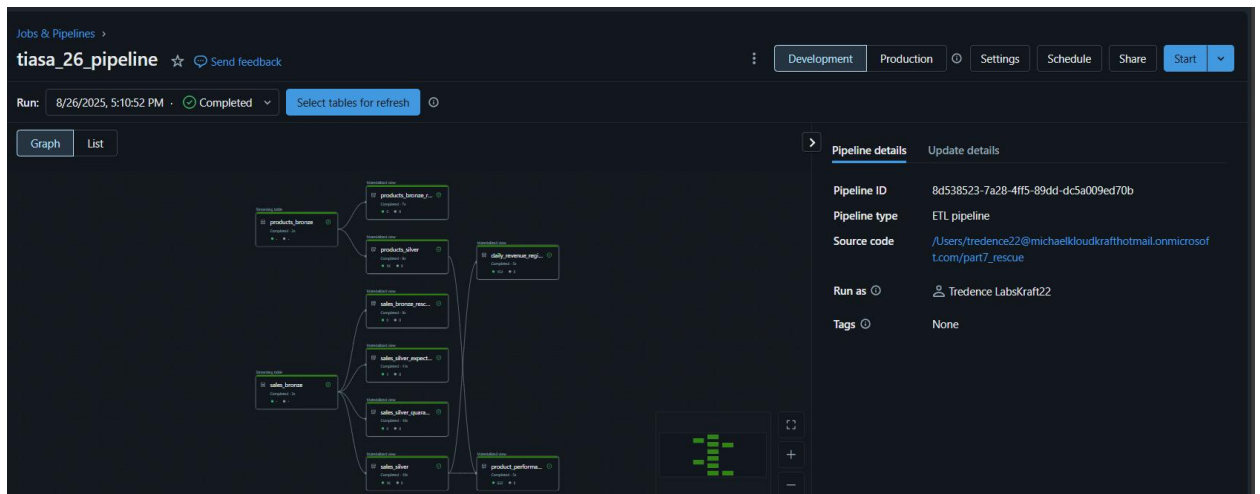
```
            F.sum((F.col("quantity").cast("int") <= 0).cast("int")).alias("bad_quantity_rows"),
```

```
            F.sum(F.col("_rescued_data").isNotNull().cast("int")).alias("rescued_rows")
```

```
        )
```

```
    )
```

Pipeline after troubleshooting



PART 6: Time Travel

2 minutes ago (8s) 1

```
%sql
-- Run DESCRIBE HISTORY on the physical bronze table
DESCRIBE HISTORY tiasa_catalog4.bronze.sales_bronze;
```

(1) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [version: long, timestamp: timestamp ... 13 more fields]

	version	timestamp	userId	userName	operation	operationParameters
1	3	2025-08-26T11:18:18.000+00:...	140943146098555	tredence22@michaelkoudkrafthotmail.onmicrosoft.co...	DELETE	> {"predicate":"(\\(region#4943 = North)\\)"}]
2	2	2025-08-26T09:37:48.000+00:...	140943146098555	tredence22@michaelkoudkrafthotmail.onmicrosoft.co...	STREAMING UPDATE	> {"outputMode":"Append","queryId":"f8de5f77-48...
3	1	2025-08-26T09:37:19.000+00:...	140943146098555	tredence22@michaelkoudkrafthotmail.onmicrosoft.co...	DLT SETUP	> {"pipelineId":"8d538523-7a28-4ff5-89dd-dc5a00...
4	0	2025-08-26T09:36:58.000+00:...	140943146098555	tredence22@michaelkoudkrafthotmail.onmicrosoft.co...	CREATE TABLE	> {"partitionBy":"[]","clusterBy":"[]","description":"nu...

4 rows | 7.89s runtime Refreshed 2 minutes ago

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

2 minutes ago (4s) 2

```
%sql
-- Query the data as it was in its first version (version 0)
SELECT * FROM tiasa_catalog4.bronze.sales_bronze VERSION AS OF 0;
```

```
-- Let's say the latest version is 1. This command will create version 2.
DELETE FROM tiasa_catalog4.bronze.sales_bronze WHERE region = 'North';
```

(2) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [num_affected_rows: long]

	num_affected_rows
1	0

1 row | 19.35s runtime

This result is stored as `_sqldf` and can be used in other Python and SQL cells.

3 minutes ago (31s) 4

```
%sql
-- Find records that were in version 1 but are missing from the current version
SELECT * FROM tiasa_catalog4.bronze.sales_bronze VERSION AS OF 1
EXCEPT
SELECT * FROM tiasa_catalog4.bronze.sales_bronze;
```

(3) Spark Jobs

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [sale_id: integer, product_id: integer ... 6 more fields]

sale_id	product_id	region	store_id	quantity	sales_amount	sale_date	_rescued_data
---------	------------	--------	----------	----------	--------------	-----------	---------------

+ Code+ textAssistant

Last execution failed5

12345

%sqlINSERT INTO tiasa_catalog4.bronze.sales_bronzeSELECT * FROM tiasa_catalog4.bronze.sales_bronze VERSION AS OF 1WHERE region = 'North';

Internal error. Attach your notebook to a different compute or restart the current compute.

Diagnose errorAssistant Quick Fix: ON

PART 7:

Creating jobs

databricks

Search data, notebooks, recents, and more...CTRL + P

Jobs & Pipelines

New Job Aug 26, 2025, 05:25 PMLakeflow Jobs UI: OFF

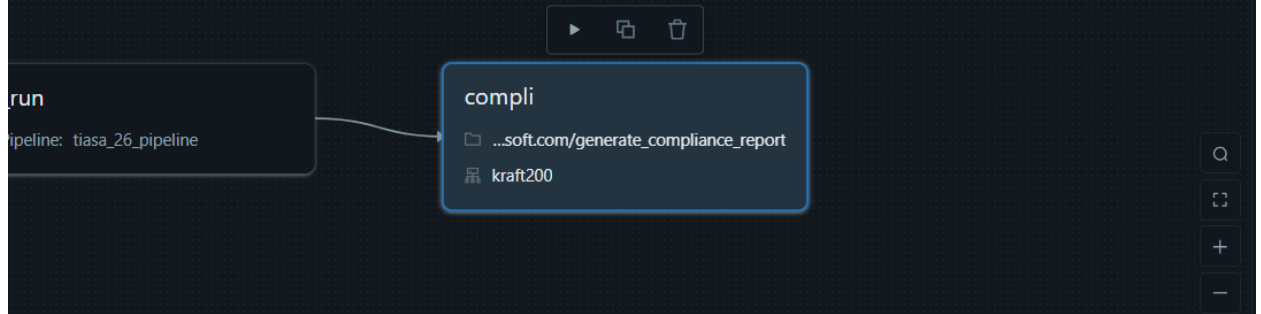
RunsTasks

Task name*Type*Pipeline*NotificationsRetriesMetric thresholds

Schedules & Triggers

Trigger StatusActivePausedTrigger typeScheduledSchedule typeSimpleAdvancedScheduleEvery 5Minutesfrom4 minutes past the hourShow cron syntaxThis schedule will frequently trigger a new job run even if the previous job run is still running. Continuous jobs are better suited to ensure that there is always an active job. Learn moreCancelSave

New Job Aug 26, 2025, 05:25 PM ☆ Lakeflow Jobs UI: OFF ▾

Runs **Tasks**

Task name* ⓘ

compli

Type*

Notebook ▾

Source* ⓘ

Workspace ▾

Path* ⓘ

...e22@michaelkloudkrafthotmail.onmicrosoft.com/generate_compliance_report ▾ 🔗

Cluster* ⓘ

kraft200 8 GB · 4 Cores · DBR 16.4 LTS · Spark 3.5.2 · Scala 2.12 🔗 ▾

ⓘ Jobs running on all-purpose clusters are considered all-purpose compute. [Learn more](#)

Depends on

dlt_run X ▾

Run if dependencies ⓘ

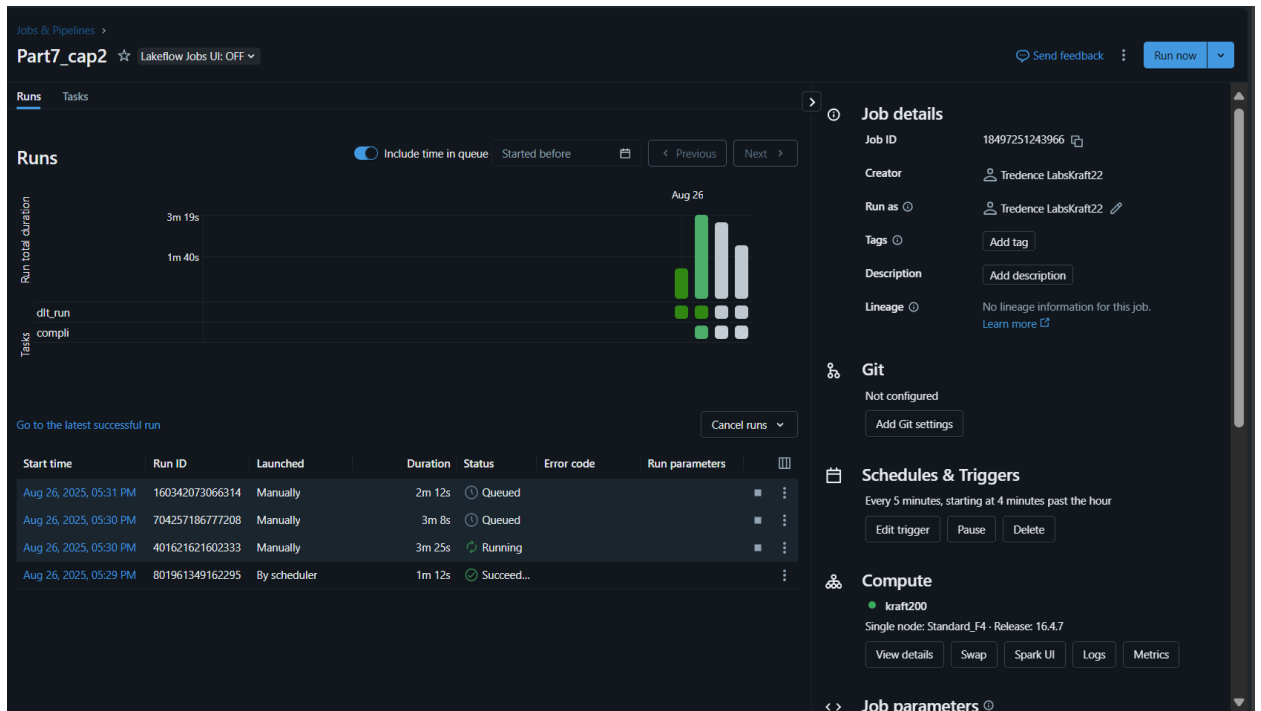
All succeeded ▾

Dependent libraries ⓘ

+ Add

Cancel

Create task



PART 8: POWER BI

Not working for me

Compute

All-purpose compute Job compute SQL warehouses Vector Search Pools Policies Apps OLTP Database

Filter SQL warehouses Only my SQL warehouses Created by Size Status Type Create SQL warehouse

Status	Name	Created by	Size	Active / Max	Type
⚠	Starter Warehouse	RohitY	Small	0 / 1	Pro